

2026-09-20

ggnarrate: A Semantic Description Layer for Accessible ggplot2 Visualisations

Uwakmfon Usen Paul 
Ghana R Users Community

Executive Summary

R has several tools for generating alt text automatically, including the recently published `ggalttext` package, but all of them describe a plot's structure rather than its content: which variables sit on which axis, not what the data actually shows. A typical output reads “a plot with mpg on the x-axis and disp on the y-axis” which is accurate, but no more informative to a screen reader user than the plot's own axis labels read aloud. Using Lundgard and Satyanarayan's four-level model of accessible description, every existing R tool operates at Level 1. None reach Level 2 (statistical relations) or Level 3 (the takeaway a sighted reader would draw).

`ggnarrate` a semantic description layer for `ggplot2` visualisations: a package that reads a plot object directly and generates a structured, statistically grounded description such as trend, outliers, spread, clustering, rather than a restatement of its axes. It integrates directly with `knitr`, `Quarto` and `Shiny`, so authors get usable alt text without writing extra code.

The scope include the five most common geoms (point, line, bar, boxplot, histogram), full test coverage, vignettes, and a documented compatibility strategy for keeping pace with `ggplot2`'s own evolution. Base R graphics and multilingual output are identified as next steps but are kept out of this initial release to keep the deliverable realistic within a 16-week, \$8,500 budget.

The project is led by Uwakmfon Usen Paul, with Francis Mensah and Samuel Tandoh of the Ghana R Users Community contributing to the proposal's scope and milestone structure.

Signatories

Project team

Uwakmfon Usen Paul, **MLS**: R Programming Instructor at the Ghana R Users Community
GitHub: <https://github.com/DevWebWacky>

I will be the sole developer delivering this project. I am proficient in R and Python. I have so far built and maintain 4 R packages and 1 python package, two (R) of which has been published in CRAN, one pending final approval, and one in development stage. Below are the packages:

- **triageR** (CRAN): a clinical prediction modelling package aligned with TRIPOD+AI reporting standards, including an optional AI-assisted layer for method recommendation.
- **statease** (CRAN): runs statistical tests and generates plain-English explanations of the results automatically.
- **rgrind** (pending CRAN approval): a gamified R learning tool, reflecting my interest in making R more approachable.
- **cystimap** (Development stage): an R package for reproducible spatial risk mapping.
- **rpynative** (PyPI): rpynative lets Python developers load and use R packages as if they were normal Python packages. No R syntax, no manual object conversion, and unlike other R-Python bridges, rpynative supports real IDE autocomplete and rich Jupyter/Colab display for R functions.

Contributors

- **Francis Mensah**, Founder, Ghana R Users Community
- **Samuel Tandoh**, Co-founder and Executive Director, Ghana R Users Community

Consulted

- **Mr. George Kyei**, R Trainer, Ghana R Users Community.

The Problem

R's plotting ecosystem, built around **ggplot2** (Wickham 2016), has tools that describe a chart's structure automatically, but nothing that automatically describes what the data actually shows.

What the problem is

Every plot produced by **ggplot2**, and rendered into an RMarkdown report, a Quarto document or a Shiny app, is by default invisible to a screen reader beyond its title and axis labels. Some tools exist to fill this gap, but none currently describe what the data actually shows. The most recent, **ggalttext** (Barbier 2026), inspects a plot's layers, labels and facets and produces text such as "a plot with mpg on the x-axis and disp on the y-axis" which is accurate, but no more informative than reading the axis labels aloud would already be. **BrailleR** (Godfrey and Murrell 2018) takes a similar structural approach for both base R and **ggplot2** graphics. Neither reports the trend, the outliers, the clustering or the spread, the content a sighted reader takes in within a second of looking at the chart.

Who it affects

This affects every screen reader user who encounters an R-generated plot: in academic papers, government statistical reports, business dashboards built in Shiny, and public-facing data journalism.

It also affects the authors producing these outputs, most of whom have no accessibility background and no realistic way to hand-write a statistically meaningful description for every chart they generate, even when they want to.

Why it is a problem

The evidence here is not anecdotal. In a controlled study of screen reader users interacting with online data visualisations, participants extracted information 61.48% less accurately, and took 210.96% longer to do so, than sighted users working with the same charts ([Sharif et al. 2021](#)). Where alt text is present at all, it commonly repeats structural metadata rather than conveying the point of the visualisation, echoing the pattern seen in R’s own tooling ([Jumping Rivers 2023](#)). The Web Content Accessibility Guidelines explicitly call for text alternatives that serve the equivalent purpose of the visual content ([W3C Web Accessibility Initiative 2018](#)), not merely a technical inventory of what is on the axes.

What solving it enables

A reliable, automatic, statistically grounded description layer means authors get usable alt text without having to write it, screen reader users get access to the actual content of a chart rather than its scaffolding, and R’s publishing tools `knitr`, `Quarto` and `Shiny`, gain a shared, dependable source of that content rather than each depending on individual authors remembering to supply it.

Existing work and previous attempts

Beyond `ggalttext` and `BrailleR`, `ggplot2` itself provides `get_alt_text()` and a `labs(alt = ...)` argument, but these only return text the author has already written by hand; the function does not generate a description from the plot’s data. `Ra11y` offers a further collection of accessibility helpers for `ggplot2`, again operating at the level of chart structure rather than content. Lundgard and Satyanarayan’s four-level model of accessible description ([Lundgard and Satyanarayan 2021](#)) shows every existing tool operating at Level 1 (structural), with none reaching Level 2 (statistical) or Level 3 (interpretive).

A 2022 community effort, run as an extension of the Minorities in R (MiR) project, attempted to crowdsource better `ggplot2` alt text using social media and Amazon Mechanical Turk ([Minorities in R \(MiR\) 2022](#)). It did not produce a general, automated solution, and the gap it identified, description of statistical content, not structure, remains open.

This proposal does not involve changes to R itself, so no letter of support from R Core is required.

The proposal

Overview

This project builds `ggnarrate`, a semantic description layer for `ggplot2` visualisations: a package that reads a plot object directly, extracts its statistical content such as trend, spread, outliers, clustering, and generates a structured, WCAG-aligned description automatically. It hooks into `knitr`, `Quarto` and `Shiny` so authors get usable alt text without writing extra code, and screen reader users get access to what a chart actually shows rather than a restatement of its axis labels.

Existing R tools stop at what Lundgard and Satyanarayan ([Lundgard and Satyanarayan 2021](#)) classify as level 1 content: chart type, axis titles, encoded properties. This project targets level 2

content including statistical relations such as trend, spread and outliers, which no existing R tool currently automates. Level 3 (the interpretive “why” behind the chart) remains the author’s to supply, and is addressed as a future direction rather than a deliverable of this grant.

Concrete actions and estimated timeline:

1. Build the extraction layer that reads a `ggplot` object’s data, mapped aesthetics and geom type.
2. Build a statistical summarisation module per geom type (point, line, bar, boxplot, histogram).
3. Build a text-generation layer that turns those statistics into plain-language description.
4. Add integration hooks for `knitr`’s `fig.alt`, Quarto, and Shiny’s `renderPlot()`.
5. Document, test, and submit to CRAN.

Detail

Minimum Viable Product

The smallest version that delivers real value is support for a single geom, `geom_point()`, producing a description that states direction and strength of association and flags any clear outliers, wired into `knitr`’s `fig.alt` chunk option. That alone would already exceed what any existing R tool produces, since none currently report statistical relationships. Everything beyond that (additional geoms, Quarto/Shiny hooks, Level 3 description) is an extension of this same core, not a separate build.

Architecture

The pipeline runs in four stages, each independently testable:

1. **Extraction:** reads the `ggplot2` object’s data, mapped aesthetics, and geom type.
2. **Semantic analysis:** computes trend, outliers, spread and clustering, per geom type.
3. **Structured description:** turns those statistics into a plain-language description.
4. **Output adapters:** one per publishing target: `knitr` (`fig.alt`), Quarto (HTML/PDF), and Shiny (`renderPlot()` alt attribute).

Adding a new geom later means writing one new module in stage 2; a change to how `knitr`, Quarto or Shiny handle alt text means updating one adapter in stage 4, not touching the rest of the pipeline.

Assumptions

- **ggplot2’s internal object structure remains introspectable in roughly its current form:** If a future `ggplot2` release changes how plot data and aesthetics are stored internally, the extraction layer would need updating. This is judged low-risk within the grant period, since `ggplot2`’s object model has been stable for years, but it is the single biggest long-term maintenance dependency and is addressed directly below.
- Statistical summaries computed from the data may not generalise cleanly to more complex plots (log scales, stacked geoms), which are why those are out of scope for this cycle.
- **A text description, however well generated, cannot fully replace the “why” behind a plot:** the author’s own communicative intent. `ggnarrate` is designed to produce a strong default, with the alt text always overridable by the author, rather than claiming to solve accessibility on its own.

External dependencies

- `ggplot2` (core dependency, `ggnarrate` will read `ggplot2` objects directly)
- `knitr`, for the `fig.alt` integration hook
- Quarto's rendering pipeline, for HTML/PDF alt-text output
- `shiny`, for the `renderPlot()` integration hook
- No external services, APIs, or paid infrastructure are required, the package is self-contained and works entirely offline

Project plan

Start-up phase

Work begins immediately on grant approval, with the following in place before the coding phase:

- **Collaboration platform:** a public GitHub repository under the developer's account, structured as a standard R package from day one (`R/`, `tests/`, `vignettes/`, `man/`), with issues enabled for anyone in the community to raise problems or suggest geom support.
- **Licence:** MIT, matching the licensing convention used by `ggplot2` itself and the majority of its extension packages, to keep the package as unrestricted and easy to adopt as possible.
- **Reporting framework:** a short written progress update posted to the repository's README and to the R Consortium at the end of each milestone.

Technical delivery

Weeks	Deliverable
1 to 3	Extraction layer: reads a <code>ggplot2</code> object's data, mapped aesthetics, and geom type. Unit tests against a fixed set of known plots.
4 to 7	Statistical summarisation module covering point, line, bar, boxplot and histogram geoms, with tests against datasets with known statistical properties.
8 to 10	Text-generation layer converting statistics into plain-language description, reviewed against WCAG alt-text guidance.
11 to 12	Integration hooks: <code>knitr</code> 's <code>fig.alt</code> , Quarto, and Shiny's <code>renderPlot()</code> .
13 to 14	Vignettes, full documentation, and internal review against real screen reader software (NVDA/VoiceOver).
15	CRAN submission and checks.
16	Buffer for CRAN reviewer feedback and any required fixes.

Other aspects

The package’s usefulness depends on people actually finding and adopting it, so this isn’t left until the end:

- Announcement post: published on the R Consortium blog once the MVP is working, to get early feedback before the full geom set is built.
- Delivery blog post: a second post on completion, covering the finished package, worked examples, and how to adopt it in existing `knitr`/Quarto/Shiny projects.
- Social media: share via my personal and the Ghana R Users Community’s channels/meetups/training sessions, and R-focused spaces such as R-bloggers and the `#rstats` community, at both the announcement and delivery stages.
- ISC meetings: I would be happy to present progress at ISC meetings if invited, particularly around the midpoint (week 8) when the statistical summarisation module is complete and demonstrable.
- UseR!: no UseR! talk is planned within the grant period itself, given the timeline, but a submission for the following year’s conference is a realistic follow-on once the package has real-world usage to report on.

Budget & funding plan

Total requested: **\$8,500 USD**

Milestone 1: Core pipeline (Weeks 1 - 7) - \$3,800

Work: Extraction layer and statistical summarisation module for all five target geoms, with unit tests.

Outcome: A working internal pipeline that can take a `ggplot2` object and produce statistically accurate summaries, verifiable by the ISC via the public repository and test suite.

Milestone 2: Text generation and integration (Weeks 8 - 12) - \$2,700

Work: Text-generation layer, WCAG-aligned review, and integration hooks for `knitr`, Quarto and Shiny.

Outcome: A usable package producing real alt text end-to-end inside actual R Markdown/Quarto/Shiny workflows.

Milestone 3: Documentation, testing, and CRAN submission (Weeks 13 - 16) - \$2,000

Work: Vignettes, full documentation, screen reader testing, CRAN submission and any resulting fixes.

Outcome: a CRAN-submitted package with complete documentation, ready for public use.

Success

Definition of done

The project is done when:

- `ggnarrate` covers all five target geoms (point, line, bar, boxplot, histogram) and generates statistically grounded descriptions, trend, spread, outliers, clustering — rather than structural metadata alone.
- It integrates with `knitr`’s `fig.alt`, Quarto, and Shiny’s `renderPlot()`, so authors get working alt text inside real documents and apps without extra manual steps.

- It has been submitted to CRAN, passes CRAN’s checks, and is either accepted or awaiting review within the grant period.
- Sample output has been tested against real screen reader software (NVDA or VoiceOver), not just visually inspected by the developer.
- The package has full unit test coverage for its extraction and summarisation logic, and vignettes covering each supported geom.

Measuring success

Progress is checked against concrete, verifiable markers at each stage rather than a single pass/fail at the end:

- **Week 7:** the core pipeline correctly summarises a fixed benchmark set of test plots with known statistical properties (known trend direction, known outliers), checkable by anyone against the public test suite.
- **Week 12:** a plot embedded in a real Quarto document or Shiny app produces alt text that a screen reader actually announces correctly this can be checked by running the example and listening to it.
- **Week 14:** at least one person outside the project, ideally a screen reader user or someone from the accessibility community, such as the Ghana R Users Community contacts consulted on this proposal — has reviewed sample output and confirmed it’s a genuine improvement over no alt text or existing Level 1 tools.
- **Week 16:** CRAN submission with a clean R CMD check.
- **Post-completion:** downloads, GitHub stars/issues, and whether knitr, Quarto or Shiny’s own documentation ends up recommending it.

Future work

Within this grant, the scope is deliberately limited to `ggplot2` and five common geoms. Natural extensions, for the developer or the wider community to pick up:

- Base R graphics support.
- Additional and composite geoms: faceted plots, multiple overlaid geoms, log-scaled axes, each of which needs its own summarisation logic.
- Author-editable descriptions with statistical scaffolding retained, letting an author add the “why” behind a plot (the true Level 3 content only a human can supply) while keeping the automatically generated statistical facts intact underneath, rather than the two being mutually exclusive.

Barbier, Joseph. 2026. *Ggalttext: Make 'Ggplot2' Fully Accessible by Generating Alternative Text*. <https://CRAN.R-project.org/package=ggalttext>.

Godfrey, A. Jonathan R., and Paul Murrell. 2018. *BrailleR: Improved Access for Blind Users*. <https://ajrgodfrey.github.io/BrailleR/>.

Jumping Rivers. 2023. “Alt Text in r: Plots, Reports, and Shiny.” <https://www.jumpingrivers.com/blog/accessibility-alt-text-in-r/>.

- Lundgard, Alan, and Arvind Satyanarayan. 2021. “Accessible Visualization via Natural Language Descriptions: A Four-Level Model of Semantic Content.” *IEEE Transactions on Visualization and Computer Graphics* 28 (1): 1073–83. <https://doi.org/10.1109/TVCG.2021.3114770>.
- Minorities in R (MiR). 2022. “Open Innovation to Improve Alt-Text in Ggplot.” https://rstudio-pubs-static.s3.amazonaws.com/893428_e587a4f59cb6497aa8c1bfb8c9e36ef0.html.
- Sharif, Ather, Sanjana S. Chintalapati, Jacob O. Wobbrock, and Katharina Reinecke. 2021. “Understanding Screen-Reader Users’ Experiences with Online Data Visualizations.” *Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS ’21)*. <https://doi.org/10.1145/3441852.3471202>.
- W3C Web Accessibility Initiative. 2018. “Web Content Accessibility Guidelines (WCAG) 2.1.” <https://www.w3.org/WAI/standards-guidelines/wcag/>.
- Wickham, Hadley. 2016. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. <https://ggplot2.tidyverse.org>.